The next listing shows the same `IpAddressMiddleware` you created using the convention-based approach, refactored to implement `IMiddleware`.

Listing 2.16 Refactoring `IpAddressMiddleware` to implement `IMiddleware`

```
public class IpAddressMiddleware : IMiddleware                          ❶
{
    private ILogger<IpAddressMiddleware> _logger;
    public IpAddressMiddleware(ILogger<IpAddressMiddleware> logger)
        => _logger = logger;                                            ❷

    public async Task InvokeAsync(HttpContext context, RequestDelegate next)❸
    {
        var ipAddress = context.Connection.RemoteIpAddress;
        _logger.LogInformation($"Visitor is from {ipAddress}");
        await next(context);
    }
}
```

❶ The middleware class implements the IMiddleware interface.

❷ Dependencies are injected into the constructor.

❸ InvokeAsync takes an HttpContext and a RequestDelegate as parameters.

The `InvokeAsync` is very similar to the one that was written using the conventions-based approach, except that this time the parameters are an `HttpContext` and a `RequestDelegate`. Any services the class depends on are injected via the middleware class's constructor, so fields are required to hold instances of the injected service.

This middleware is registered in exactly the same way as the convention-based example: via the `UseMiddleware` methods or an extension method. But an additional step is also required for `IMiddleware`-based components: they must also be registered with the application's service container. You will learn much more about services and dependency